

CUDA MODE:

Lecture 1

Mark Saroufim

CPU profiling: where does my GPU program spend time, and why?

PyTorch operation → CUDA kernel launch → GPU execution

Logistics

└ Application Level

└ Kernel Level

└ Hardware Level

Hosts: Andreas Köpf, Thomas Viehmann, Mark Saroufim

1 per 2 week on a CUDA topic: textbook chapter, pair programming session or project

Target audience is torch programmers tired of CUDA tutorial hell

Textbook: [Programming Massively Parallel Processors](#)

Additional resources here in [resource-stream](#)

All communication will happen on our [Discord](#)

Sessions will be recorded on <https://www.youtube.com/@CUDAMODE>

“Don't optimize what merely looks slow
└ profile to identify what actually controls runtime”

Goal of Lecture 1

1. Integrate a CUDA kernel inside a pytorch program
2. Learn how to profile it

write kernel → verify correctness → benchmark
measure again ← change kernel ← identify bottleneck ← profile

Most of the code is here

<https://github.com/msaroufim/cudamodule1>

I believe thing I see



Start with something simple

```
*****
Profiling torch.square
*****
STAGE:2024-01-12 20:00:20 2559:2559 ActivityProfilerController.cpp:312] Completed Stage: Warm Up
STAGE:2024-01-12 20:00:20 2559:2559 ActivityProfilerController.cpp:318] Completed Stage: Collection
STAGE:2024-01-12 20:00:20 2559:2559 ActivityProfilerController.cpp:322] Completed Stage: Post Processing

-----
      Name      Self CPU %      Self CPU    CPU total %    CPU total    CPU time avg    Self CUDA    Self CUDA %    CUDA total    CUDA time avg    # of Calls
-----
      aten::square      0.58%      20.000us      3.34%      115.000us      115.000us      17.000us      0.48%      3.506ms      3.506ms      1
      aten::pow         2.09%      72.000us      2.76%      95.000us      95.000us      3.480ms      99.26%      3.489ms      3.489ms      1
      aten::result_type  0.06%      2.000us      0.06%      2.000us      2.000us      5.000us      0.14%      5.000us      5.000us      1
      aten::to          0.00%      0.000us      0.00%      0.000us      0.000us      4.000us      0.11%      4.000us      4.000us      1
      cudaLaunchKernel  0.61%      21.000us      0.61%      21.000us      21.000us      0.000us      0.00%      0.000us      0.000us      1
      cudaDeviceSynchronize 96.66%      3.332ms      96.66%      3.332ms      3.332ms      0.000us      0.00%      0.000us      0.000us      1
-----

Self CPU time total: 3.447ms
Self CUDA time total: 3.506ms

*****
Profiling a * a
*****
STAGE:2024-01-12 20:00:20 2559:2559 ActivityProfilerController.cpp:312] Completed Stage: Warm Up
STAGE:2024-01-12 20:00:20 2559:2559 ActivityProfilerController.cpp:318] Completed Stage: Collection
STAGE:2024-01-12 20:00:20 2559:2559 ActivityProfilerController.cpp:322] Completed Stage: Post Processing

-----
      Name      Self CPU %      Self CPU    CPU total %    CPU total    CPU time avg    Self CUDA    Self CUDA %    CUDA total    CUDA time avg    # of Calls
-----
      aten::mul         0.97%      33.000us      1.41%      48.000us      48.000us      3.455ms      100.00%      3.455ms      3.455ms      1
      cudaLaunchKernel  0.44%      15.000us      0.44%      15.000us      15.000us      0.000us      0.00%      0.000us      0.000us      1
      cudaDeviceSynchronize 98.59%      3.363ms      98.59%      3.363ms      3.363ms      0.000us      0.00%      0.000us      0.000us      1
-----

Self CPU time total: 3.411ms
Self CUDA time total: 3.455ms

*****
Profiling a ** 2
*****
STAGE:2024-01-12 20:00:20 2559:2559 ActivityProfilerController.cpp:312] Completed Stage: Warm Up
STAGE:2024-01-12 20:00:20 2559:2559 ActivityProfilerController.cpp:318] Completed Stage: Collection
STAGE:2024-01-12 20:00:20 2559:2559 ActivityProfilerController.cpp:322] Completed Stage: Post Processing

-----
      Name      Self CPU %      Self CPU    CPU total %    CPU total    CPU time avg    Self CUDA    Self CUDA %    CUDA total    CUDA time avg    # of Calls
-----
      aten::pow         1.37%      47.000us      1.69%      58.000us      58.000us      3.461ms      99.74%      3.470ms      3.470ms      1
      aten::result_type  0.03%      1.000us      0.03%      1.000us      1.000us      5.000us      0.14%      5.000us      5.000us      1
      aten::to          0.00%      0.000us      0.00%      0.000us      0.000us      4.000us      0.12%      4.000us      4.000us      1
      cudaLaunchKernel  0.29%      10.000us      0.29%      10.000us      10.000us      0.000us      0.00%      0.000us      0.000us      1
      cudaDeviceSynchronize 98.31%      3.378ms      98.31%      3.378ms      3.378ms      0.000us      0.00%      0.000us      0.000us      1
-----

Self CPU time total: 3.436ms
Self CUDA time total: 3.470ms
```

```
9  def time_pytorch_function(func, input):
10     # CUDA IS ASYNC so can't use python time module
11     start = torch.cuda.Event(enable_timing=True)
12     end = torch.cuda.Event(enable_timing=True)
13
14     # Warmup
15     for _ in range(5):
16         func(input)
17
18     start.record()
19     func(input)
20     end.record()
21     torch.cuda.synchronize()
22     return start.elapsed_time(end)
23
24 b = torch.randn(10000, 10000).cuda()
25
26 def square_2(a):
27     return a * a
28
29 def square_3(a):
30     return a ** 2
31
32 time_pytorch_function(torch.square, b)
33 time_pytorch_function(square_2, b)
34 time_pytorch_function(square_3, b)
35
36 print("=====")
37 print("Profiling torch.square")
38 print("=====")
39
```

PyTorch profiler

Memcpy HtoD (Pageable -> Device) *→ copying data from host to device (CPU) (GPU)*

- Host to device copy
- Pageable memory is on host but can be copied freely in out of RAM



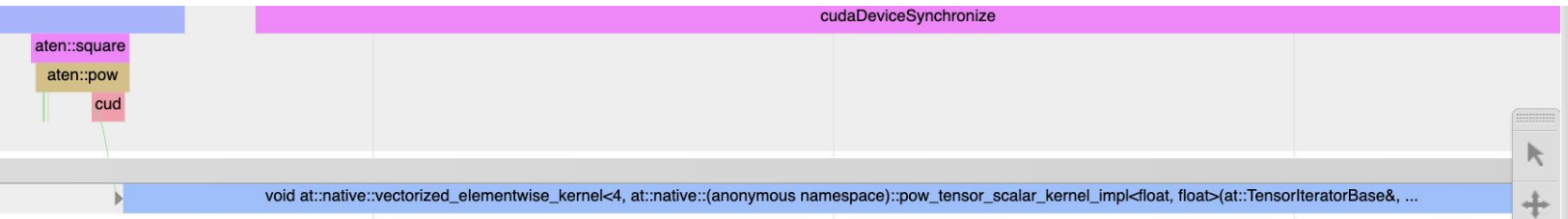
What can we learn?

Aten::square is a call to aten::pow *implemented using* *launches*

A cuda kernel gets launched called native_vectorized_elementwise_kernel<4, ..>

4 is the number of blocks

<https://github.com/pytorch/pytorch/blob/main/caffe2/utils/math/elementwise.cu>



Custom cpp extensions

```
hello_load_inline.py > ...  
1  import torch  
2  from torch.utils.cpp_extension import load_inline  
3  
4  cpp_source = """  
5      std::string hello_world() {  
6          return "Hello World!";  
7      }  
8      """  
9  
10 my_module = load_inline(  
11     name='my_module',  
12     cpp_sources=[cpp_source],  
13     functions=['hello_world'],  
14     verbose=True  
15 )  
16  
17 print(my_module.hello_world())
```

integrating C/C++ code into
a python code
so that we can write
CUDA kernels inline python
(CUDA = C/C++)

Codegen

```
#include <torch/extension.h>
```

```
std::string hello_world() {  
    return "Hello World!";  
}
```

```
PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {  
    m.def("hello_world", torch::wrap_pybind_function(hello_world), "hello_world");  
}
```


How to run a CUDA kernel from pytorch

https://github.com/msaroufim/cudamodelecture1/blob/main/load_inline.py

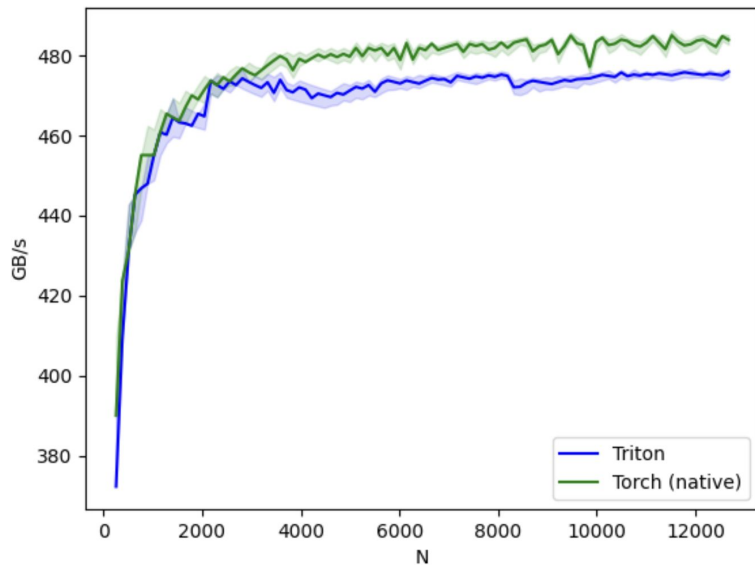
numba

<https://gist.github.com/msaroufim/6673c9e5c0c3d58740472601eac6d4df>

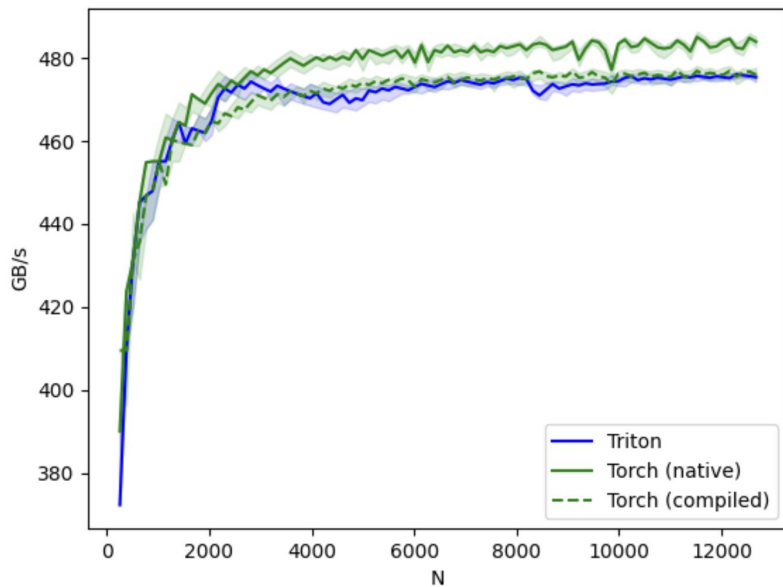
Integrate a triton kernel → not gonna generate / write CUDA
but generate PTX kernel / code (CUDA assembly)

```
27 def square(x):
28     n_rows, n_cols = x.shape
29     # The block size is the smallest power of two greater than the number of columns in `x`
30     BLOCK_SIZE = triton.next_power_of_2(n_cols)
31     # Another trick we can use is to ask the compiler to use more threads per row by
32     # increasing the number of warps (`num_warps`) over which each row is distributed.
33     # You will see in the next tutorial how to auto-tune this value in a more natural
34     # way so you don't have to come up with manual heuristics yourself.
35     num_warps = 4
36     if BLOCK_SIZE >= 2048:
37         num_warps = 8
38     if BLOCK_SIZE >= 4096:
39         num_warps = 16
40     # Allocate output
41     y = torch.empty_like(x)
42     # Enqueue kernel. The 1D launch grid is simple: we have one kernel instance per row o
43     # f the input matrix
44     square_kernel[(n_rows, )](
45         y,
46         x,
47         x.stride(0),
48         y.stride(0),
49         n_cols,
50         num_warps=num_warps,
51         BLOCK_SIZE=BLOCK_SIZE,
52     )
53     return y
54
```

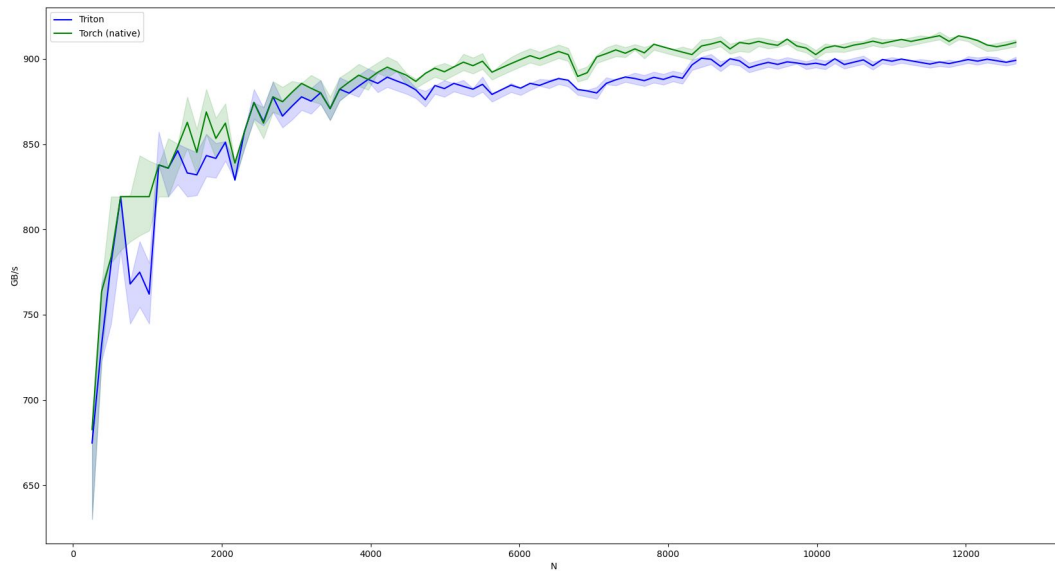
Triton kernel on A10G



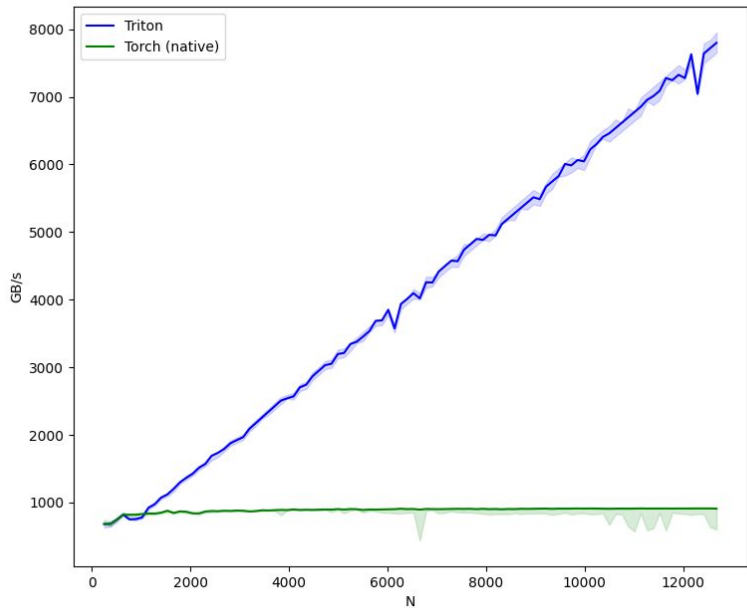
torch.compile



Results on 4090



After fixing the block size to 1024



Triton has a debugger now

★ correctness first
performance second

```
triton.jit(interpret=True)
```

Almost everything is a WrappedTensor so inspect variables with `var_name.tensor`

<https://gist.github.com/msaroufim/f849df30687708782e0269c4b42264b1>

Look at **PTX** → low level CPU instructions generated before final machine code

https://github.com/msaroufim/cudamodelecture1/blob/main/square_kernel.ptx

8 registers with a self multiplication for input

8 registers for output

```
.loc 1 19 26
```

```
mul.f32    %f9, %f1, %f1;
```

```
mul.f32    %f10, %f2, %f2;
```

```
mul.f32    %f11, %f3, %f3;
```

```
mul.f32    %f12, %f4, %f4;
```

```
mul.f32    %f13, %f5, %f5;
```

```
mul.f32    %f14, %f6, %f6;
```

```
mul.f32    %f15, %f7, %f7;
```

```
mul.f32    %f16, %f8, %f8;
```

Cheat: Generate a triton kernel

Triton $\rightarrow$ Pyx $\rightarrow$ GPU machine code $\rightarrow$ GPU exec.

TORCH_LOGS="output_code" python compile_square.py

`torch.compile(torch.square))` $\rightarrow$ auto. generates optimized kernel

```
import triton
import triton.language as tl
from torch._inductor.ir import ReductionHint
from torch._inductor.ir import TileHint
from torch._inductor.triton_heuristics import AutotuneHint, pointwise
from torch._inductor.utils import instance_descriptor
from torch._inductor import triton_helpers

@pointwise(size_hints=[2097152], filename=__file__, meta={'signature': {0: '*fp32', 1: '*fp32', 2:
@triton.jit
def triton_(in_ptr0, out_ptr0, xnumel, XBLOCK : tl.constexpr):
    xnumel = 1423763
    xoffset = tl.program_id(0) * XBLOCK
    xindex = xoffset + tl.arange(0, XBLOCK)[: ]
    xmask = xindex < xnumel
    x0 = xindex
    tmp0 = tl.load(in_ptr0 + (x0), xmask)
    tmp1 = tmp0 * tmp0
    tl.store(out_ptr0 + (x0), tmp1, xmask)
```

ncu profiler

→ most used profiler

ncu python train.py

ncu --set full -o output \$(which python) train.py

https://github.com/msaroufim/cudamodelecture1/blob/main/ncu_logs

Contains actionable hints like

OPT This kernel grid is too small to fill the available resources on this device, resulting in only 0.4 full waves across all SMs. Look at Launch Statistics for more details.

* PyTorch profiler

↳ find the slowest kernel

* ncu profiler

↳ understand why that kernel is slow

Zoom in

Tail effect + Achieved occupancy 70%: Try padding (We can control)

Long scoreboard stalls: coalesce, use shared memory (Controlled by Triton : ()

	CUDA	TRITON
Memory Coalescing	Manual	Automatic
Shared Memory Management	Manual	Automatic
Scheduling (Within SMs)	Manual	Automatic
Scheduling (Across SMs)	Manual	Manual

Compiler optimizations in CUDA vs Triton.

Activities

NVIDIA Nsight Compute

Jan 11 22:18

NVIDIA Nsight Compute

FileConnectionDebugProfileToolsWindowHelp

ConnectDisconnectTerminateProfile Kernel

Some metric columns are hidden by default. Right-click any column header and use the column chooser to show them.

Project Explorer

Search project...Default Project

output/scalapw.x

Untitled 1.x

Eng:Details

Result

1-576-square_kernel_0d1d234

12.38 usecond

25,830

20

0-NVIDIA GeForce RTX 4090

2.08 cycle/usecond

8.9

[10573] python3.10

Copy as Image

Current

576-square_kernel_0d1d234 [823, 1, 1]x[128, 1, 1]

Summary of the activity or the screwdrivers issuing instructions. Each screwdriver maintains a pool or warps that it can issue instructions for. The upper bound or warps in the pool (inactive warps) is limited by the launch configuration. On every cycle each screwdriver checks the state of the associated warps in the pool (active warps). Active warps that are not stalled (inactive warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]

8.56

No Eligible [%]

95.40

Eligible Warps Per Scheduler [warp]

0.11

One or More Eligible [%]

4.60

Issued Warp Per Scheduler

0.05

Issue Slot Utilization

Est. Local Speedup: 19.04%

Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 21.7 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this kernel allocates an average of 8.56 active warps per scheduler, but only an average of 0.11 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, reduce the time the active warps are stalled by inspecting the top stall reasons on the [Warp Stall Sampling](#) and [Warp Stall Sampling](#) sections.

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]

186.27

Avg. Active Threads Per Warp

32

Warp Cycles Per Executed Instruction [cycle]

194.84

Avg. Not Predicated Off Threads Per Warp

30.66

Long Scoreboard Stalls

Est. Speedup: 19.04%

On average, each warp of this kernel spends 99.4 cycles being stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, texture) operation. Find the instruction producing the data being waited upon to identify the culprit. To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture, attempt to increase cache hit rates by increasing data locality (coalescing), or by changing the cache configuration. Consider moving frequently used data to shared memory. This stall type represents about 53.3% of the total average of 186.3 cycles between issuing two instructions.

Warp Stall

Check the [Warp Stall Sampling \(all samples\)](#) table for the top stall locations in your source based on sampling data. The [Kernel Profiling Guide](#) provides more details on each stall reason.

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]

488,564

Avg. Executed Instructions Per Scheduler [inst]

954.23

Issued Instructions [inst]

511,042

Avg. Issued Instructions Per Scheduler [inst]

998.13

FP32 Non-Fused Instructions

Est. Speedup: 0.73%

This kernel executes 0 fused and 58336 non-fused FP32 instructions. By converting pairs of non-fused instructions to their [fused](#), higher-throughput equivalent, the achieved FP32 performance could be increased by up to 50% (relative to its current performance). Check the Source page to identify where this kernel executes FP32 instructions.

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size

1,823

Execution Cache Configuration

CachePreference

0

Registers Per Thread [register/thread]

20

Static Shared Memory Per Block [byte/block]

0

Block Size

128

Dynamic Shared Memory Per Block [byte/block]

0

Threads [thread]

233,344

Driver Shared Memory Per Block [kbyte/block]

1.02

Waves Per SM

1.19

Shared Memory Configuration Size [kbyte]

32.77

Tail Effect

Est. Speedup: 50.00%

A wave of thread blocks is defined as the maximum number of blocks that can be executed in parallel on the target GPU. The number of blocks in a wave depends on the number of multiprocessors and the theoretical occupancy of the kernel. This kernel launch results in 1 full waves and a partial wave of 286 thread blocks. Under the assumption of a uniform execution duration of all thread blocks, the partial wave may account for up to 50.0% of the total kernel runtime with a lower occupancy of 27.3%. Try launching a grid with no partial wave. The overall impact of this tail effect also lessens with the number of full waves executed for a grid. See the [Launch Waves Model](#) description for more details on launch configurations.

The following table lists the metrics that are key performance indicators:

Metric Name

Value

Guidance

launch_waves_per_multiprocessor

1.18683

Decrease the number of partial waves (the fractional part of the number of waves)

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]

100

Block Limit Registers [block]

21

Theoretical Active Warps per SM [warp]

48

Block Limit Shared Mem [block]

32

		Copy as Image			
Source: square_kernel_0d1d234					
Navigation: Instructions Executed					
#	Address	Source	Live ar Registers	Stall (All Samples)	ins #
19	00007faf	b5295520	LDP3.LUT_R2, R7, 0x200, R2,	9	
20	00007faf	b5295530 #IP3	LDG_E R6, [R6, 64]	9	4.23%
21	00007faf	b5295540	LDP3.LUT_R3, R7, 0x300, R2,	10	
22	00007faf	b5295550	ISETP.GE_AND R0, PT, R0, c1	10	0.04%
23	00007faf	b5295560	ISETP.GE_AND R0, PT, R0, c1	9	0.04%
24	00007faf	b5295570	ISETP.GE_AND P0, PT, R0, c1	8	0.04%
25	00007faf	b5295580	LDP3.LUT_R0, R7, 0x300, R2,	8	
26	00007faf	b5295590	CS2R R0, SR2	10	
27	00007faf	b52955a0	CS2R R10, SR2	12	
28	00007faf	b52955b0	CS2R R12, SR2	14	0.04%
29	00007faf	b52955c0	ISETP.GE_AND P0, PT, R0, c1	14	
30	00007faf	b52955d0	IMAD.MOV_U32 R14, R2, R2, R	14	
31	00007faf	b52955e0 #IP0	LDG_E R8, [R6, 64+0x200]	14	0.68%
32	00007faf	b52955f0 #IP2	LDG_E R9, [R6, 64+0x200]	14	0.42%
33	00007faf	b5295600 #IP2	LDG_E R10, [R6, 64+0x200]	14	0.35%
34	00007faf	b5295610 #IP3	LDG_E R11, [R6, 64+0x200]	14	0.45%
35	00007faf	b5295620 #IP0	LDG_E R12, [R6, 64+0x200]	14	0.25%
36	00007faf	b5295630 #IP5	LDG_E R13, [R6, 64+0x200]	14	0.38%
37	00007faf	b5295640 #IP6	LDG_E R14, [R6, 64+0x200]	14	0.45%
38	00007faf	b5295650	IMAD_R2, R15, c[0x0][0x70]	13	
39	00007faf	b5295660	PZR R16, P0, R2, 0x1 PZR R1	13	0.04%
40	00007faf	b5295670	ISETP.GE_AND R0, PT, R0, c1	13	0.04%
41	00007faf	b5295680	IMAD.WIDE_U32 R2, R17, c[0x0]	10	
42	00007faf	b5295690	IMAD.WIDE_U32 R2, R7, 0x1,	10	0.15%
43	00007faf	b52956a0	SEL R0, R0, 0x7f000000, 1PC	12	29.34%
44	00007faf	b52956b0	FMUL R7, R0, R0	13	0.04%
45	00007faf	b52956c0 #IP0	STG_E [R2, 64], R7	12	6.72%
46	00007faf	b52956d0	ISETP.NE_AND P0, PT, R15, E	11	
47	00007faf	b52956e0	SEL R0, R0, 0x7f000000, 1PC	10	5.40%
48	00007faf	b52956f0	SEL R0, R0, 0x7f000000, 1P3	10	3.55%
49	00007faf	b5295700	SEL R10, R10, 0xffff0000, 1	10	6.99%
50	00007faf	b5295710	SEL R11, R11, 0xffff0000, 1	10	0.04%
51	00007faf	b5295720	SEL R12, R12, 0xffff0000, 1	10	0.00%
52	00007faf	b5295730	SEL R13, R13, 0xffff0000, 1	10	0.00%
53	00007faf	b5295740	FMUL R15, R0, R0	11	0.04%
54	00007faf	b5295750	FMUL R0, R0, R0	10	0.04%
55	00007faf	b5295760	FMUL R0, R10, R10	11	
56	00007faf	b5295770	FMUL R11, R11, R11	10	
57	00007faf	b5295780	FMUL R7, R12, R12	11	2.15%
58	00007faf	b5295790	FMUL R13, R13, R13	10	0.00%
59	00007faf	b52957a0	SEL R14, R14, 0xffff0000, 1	10	
60	00007faf	b52957b0 #IP0	STG_E [R2, 64+0x200], R15	10	2.11%

Select a source line in an active inline function to show additional information

